

User Guide for OpenFOAM-8

*surfaceChemistryFoam: An OpenFOAM-based library for detailed
surface chemistry in reacting flow simulations*

Please cite our article if you are using this package: D. N. Nguyen, J. H. Lee, H. W. Seo, H. J. Ahn, C. S. Yoo, *surfaceChemistryFoam: An OpenFOAM-based library for detailed surface chemistry in reacting flow simulations*, Computer Physics Communications (2026)(submitted).

Contents

1	General descriptions	4
1.1	Library for surface chemistry in OpenFOAM	4
1.2	catalystFoam solver	7
1.3	surfChemkinToFoam utility	8
2	Running simulations using catalystFoam solver	8
2.1	Case file description	8
2.2	Settings at time directory	9
2.2.1	Setting for mass fraction fields of gas-phase species	9
2.2.2	Setting for site fraction fields of surface species	10
2.2.3	Setting for velocity field (U)	11
2.2.4	Setting for temperature field (T) of gas-phase species	13
2.2.5	Setting for temperature field (T.catalyst) of surface species	14
2.2.6	Setting for pressure field (p) of gas-phase species	14
2.2.7	Setting for pressure field (p.catalyst) of surface species	14
2.3	Settings at constant directory	15
2.3.1	Setting for thermophysicalProperties file	15
2.3.2	Setting for catalystThermophysicalProperties file	17
2.3.3	Setting for chemistryProperties file	17
2.3.4	Setting for surfaceChemistryProperties file	18
2.4	Settings at system directory	20
2.5	Preparing input thermo and reaction files	20
2.5.1	Preparing input files for gas-phase	20
2.5.2	Preparing input files for surface-phase	20
2.6	Running simulation using catalystFoam	24
3	Using surface chemistry library in a custom OpenFOAM solver	25
3.1	Setup for compilation	25

3.2	Creating surface chemistry object	26
3.3	Updating surface chemistry object	27
3.4	Using surface chemistry with a particle-resolved model	27

1. General descriptions

The framework is fully native OpenFOAM(OF) and comprises newly developed libraries and solvers as follows:

- libraries:
 - **thermophysicalModels** includes surface chemistry sub-library.
- solvers:
 - **catalystFoam**
- utilities:
 - **surfChemkinToFoam**

1.1. Library for surface chemistry in OpenFOAM

Figure 1 presents the class diagram of the updated **thermophysicalModels** library, incorporating the newly-developed **surfaceChemistryModels** sub-library. In the diagram, green boxes with circled numbers indicate representative classes selected via OF’s runtime selection mechanism, while yellow boxes denote newly introduced classes for surface chemistry modeling. The main components of the framework are summarized below.

- *catSpecieCoeffs* provides functionality for reading surface reaction parameters, including stoichiometric coefficients for surface-site and gas-phase species, as well as kinetic parameters such as pre-exponential factors and activation energies.
- *SurfaceReaction* is an abstract base class that defines the common interface for all surface reaction models.
- *IrreversibleSurfaceReaction* is developed from the *IrreversibleReaction* class and implements the generalized surface reaction rate formulation to evaluate rate constants for different classes of surface reactions.

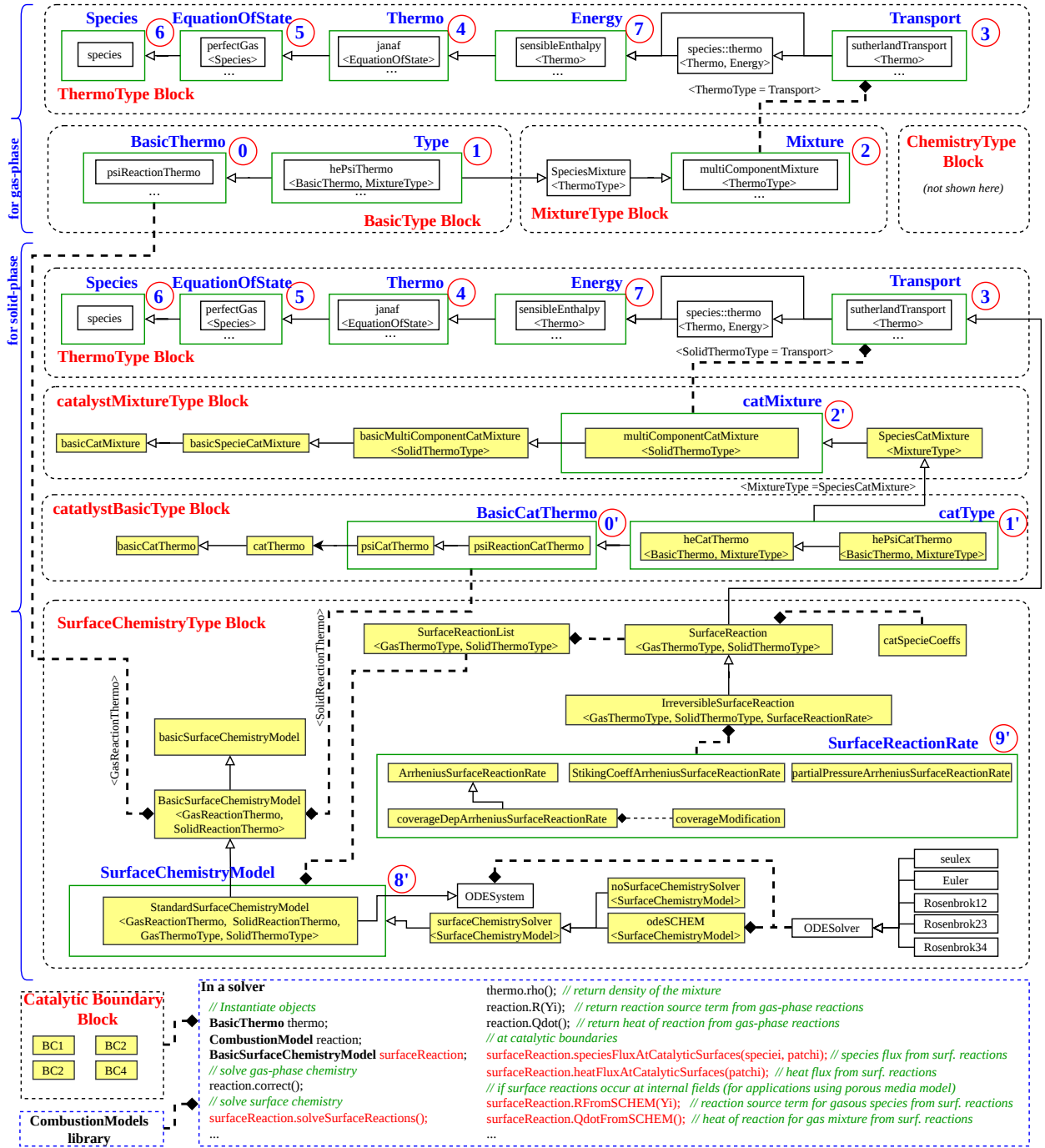


Figure 1: Updated `thermophysicalModels` library, incorporating the `surfaceChemistryModels` sub-library for modeling surface reactions in OF. Boxes with circled numbers indicate representative classes selected via run-time selection. Yellow boxes denote newly added classes for surface chemistry. Solid arrows indicate inheritance relationships, while dashed arrows represent usage relationships between classes.

- Classes in the box marked by ⑨ correspond to specific surface reaction rate constant models selected through the surface reaction mechanism input file, from which the detailed coefficients required for each reaction type are obtained.
- *SurfaceReactionList* is developed from the *ReactionList* class, which stores and manages all surface reactions included in the microkinetic mechanism.
- *StandardSurfaceChemistryModel* class, in the box marked by ⑧, is the main class for solving heterogeneous surface reaction kinetics coupled with thermodynamic and transport models. It is templated to support different thermo and transport packages, enabling flexible integration with various reacting flow solvers (see the user guide provided in our GitHub repository [1] for details of supported combinations of thermo and transport packages). Similar to the original *StandardChemistryModel* class used for gas-phase reactions, this class manages evaluation of net reaction rates, Jacobian assembly for stiff ODE chemistry solver, species flux (or species source terms), and heat flux (or heat of reaction term) from surface reactions (see the manuscript for details of coupling gas-phase and surface chemistry).
- *BasicSurfaceChemistryModel* is an abstract base class of the *surfaceChemistryModels* sub-library, developed from *BasicChemistryModel*. It serves as the primary interface between surface chemistry models and other OF libraries and solvers.
- Classes in *catalystMixtureType* block define mixture models for evaluation of thermophysical properties of solid parts within catalysts.
- Classes in *catalystBasicType* block are abstract base classes responsible for handling the thermophysical properties of solid mixtures within catalysts.
- Classes in the *ThermoType* block are existing OF thermophysical models that are combined to form a composite *GasThermoType* and *SolidThermoType* representing the gaseous and solid components of catalysts (see Section 2.3.1 for details of these combinations).

- Classes in *catalyticBoundary* block are responsible for handling temperature, gas-phase species mass fraction fields, site fraction fields of site species, and the correction velocity accounting for the Stefan flow effect at the catalytic surfaces. BC1–BC4 correspond to *heatFluxFromCatalyticSurfaces*, *speciesFluxFromCatalyticSurfaces*, *catalyticWall*, and *stefanFlowVelocity*, respectively. The BC1 and BC2 classes are derived from *fixedGradientFvPatchScalarField*, whereas the BC3 and BC4 are derived from the *fixedValueFvPatchField*. Detailed formulations implemented in these classes are presented in the manuscript.

1.2. *catalystFoam* solver

The **catalystFoam** solver is developed for the simulation of catalytic processes and reactive flows, capturing coupled interactions between gas-phase transport and catalytic surface reactions, including chemical kinetics, heat transfer, and mass transport. The governing equations implemented in **catalystFoam** are provided in the manuscript. Two options are available to feature the gas-phase flow.

Under the first option, the solver resolves the conservation equations for mass, momentum, energy, and species using the simple Sutherland transport model [2] under standard simplifying assumptions, including unity Schmidt and Lewis numbers and the neglect of radiative heat transfer, Dufour, and Soret effects.

In the second option, a detailed transport model based on the principles of kinetic gas theory [3] is employed to provide higher-fidelity description of transport phenomena beyond the assumptions of unity Lewis and Schmidt numbers. In this approach, the Dufour effect is neglected while the Soret effect can be optionally considered.

Switching between the two options is achieved by setting the keyword *usingDetailedTransportModel* in the *thermophysicalProperties* dictionary to either *false* or *true*, with the default value being *false*. When the detailed transport model is enabled, the Soret effect can additionally be included or neglected by setting the *soretEffect* (or *soretEffects*) keyword in the *thermophysicalProperties* dictionary to *true* or *false*, respectively. Details can be found in Section 2.3.1.

1.3. *surfChemkinToFoam* utility

Based on the original OpenFOAM `chemkinToFoam` utility, a new preprocessing utility named `surfChemkinToFoam` has been developed for heterogeneous surface chemistry. This utility automatically converts surface-chemistry input files from CHEMKIN format into OpenFOAM format. Detailed instructions for using this utility are provided in Section 2.5.2.

2. Running simulations using `catalystFoam` solver

2.1. *Case file description*

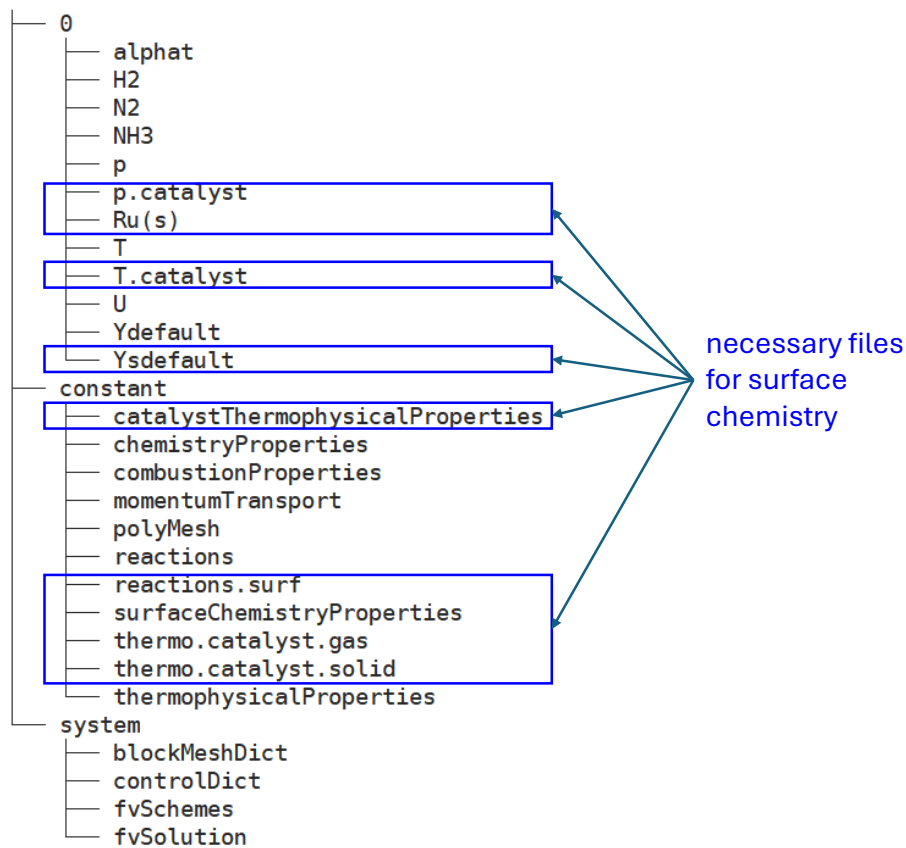


Figure 2: Case file structure of the `catalystFoam` solver, highlighting additional files required for surface chemistry.

Since `catalystFoam` is derived from the `reactingFoam` solver, its case file structure is largely identical to that of a standard `reactingFoam` case, except for several additional files required to handle surface chemistry, as illustrated in Figure 2.

In the time directory (e.g., 0), the files `p.catalyst`, `T.catalyst`, and `Ysdefault` represent the pressure, temperature, and default site-species fraction fields of the solid phase (i.e., the catalyst), respectively. The file `Ru(s)` defines the site fraction field of the Ru catalyst. The site-species name can be modified depending on the specific catalytic system being modeled. By default, all intermediate site-species fractions are initialized to zero; users may specify nonzero initial values for intermediate site species if required.

In the `constant` directory, four additional files—`catalystThermophysicalProperties`, `surfaceChemistryProperties`, `reactions.surf`, and `thermo.catalyst.solid`—are added to define surface thermodynamics and reaction mechanisms. The `system` directory remains unchanged and follows the standard setup used for `reactingFoam` cases.

In the directory `surfaceChemistryFoam-8/tutorials/ [1]`, a tutorial case featuring a three-sphere catalyst configuration, `NH3decomposition.threeBalls`, is provided to demonstrate the usage of the `catalystFoam` solver. In addition, two validation cases presented in the manuscript are provided. The first case, named `stagnationFlowReactor.Case1`, corresponds to Case 1 presented in Section 5.1.1 of the manuscript. The second case, named `annularFlowReactor`, corresponds to the validation case presented in Section 5.2 of the manuscript. Users are strongly encouraged to study those examples to gain a clear understanding of case setup and execution before applying the solver to their own practical problems.

2.2. Settings at time directory

It is important to note that, after all required files have been prepared, the boundary conditions for catalytic surfaces (i.e., boundary patches) must be specified correctly.

2.2.1. Setting for mass fraction fields of gas-phase species

For gas-phase species, the internal field and non-reacting boundary conditions can be configured in the same manner as in `reactingFoam`. For catalytic surface boundaries, how-

ever, the boundary condition must be specified as `speciesFluxFromCatalyticSurfaces`, as illustrated below (e.g., `heatedWall` is a catalytic surface):

```
/*-----* C++ *-----*\
=====
\\      / F i e l d           | OpenFOAM: The Open Source CFD Toolbox
\\      / O p e r a t i o n    | Website:  https://openfoam.org
\\      / A n d                | Version:   8
\\      / M a n i p u l a t i o n |
\*-----*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        volScalarField;
    location     "0";
    object       02;
}
// * * * * *
dimensions      [0 0 0 0 0 0 0];

internalField   uniform 0.0244595157039;

boundaryField
{
    heatedWall
    {
        type          speciesFluxFromCatalyticSurfaces;
        gradient       uniform 0.0;
        value          $internalField;
    }
    inlet
    {
        type          fixedValue;
        value          uniform 0.0244595157039;
    }
    ...
}
// ***** //
```

Do not forget to apply the same setup to the `Ydefault` file.

2.2.2. Setting for site fraction fields of surface species

For surface-phase species, the internal field can be assigned dummy values or configured in the same manner as in `reactingFoam`. However, the summation of all internal-field values must remain non-zero, since the program may terminate if all values are specified as zero, even though these fields are not directly used in catalytic simulations.

For non-reacting boundaries, the boundary conditions can likewise be assigned dummy values or configured in the same manner as in `reactingFoam`. It is recommended to specify

`zeroGradient` for all non-reacting boundaries of surface-phase species.

For catalytic surface boundaries, however, the boundary condition must be specified as `catalyticWall`, as illustrated below (e.g., `heatedWall` is a catalytic surface):

```
/*-----* C++ *-----*\
=====
\\      / F ield      | OpenFOAM: The Open Source CFD Toolbox
\\      / O peration  | Website:  https://openfoam.org
\\      / A nd        | Version:   8
\\      / M anipulation|
\*-----*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        volScalarField;
    location     "0";
    object       Rh(s).catalyst;
}
// *****
dimensions      [0 0 0 0 0 0 0];

internalField   uniform 1;

boundaryField
{
    heatedWall
    {
        type          catalyticWall;
        value          uniform 1; // this is initial surface coverage
    }
    inlet
    {
        type          zeroGradient;
    }
    ...
}
// *****
```

Do not forget to apply the same setup to the `Ysdefault` file.

2.2.3. Setting for velocity field (U)

For the velocity field U , the internal field and non-reacting boundaries can generally be configured in the same manner as in `reactingFoam`.

For catalytic wall boundaries, when the mass-flux contribution induced by the Stefan velocity is negligible, the no-slip condition is typically applied using `noSlip`, as illustrated below (e.g., `heatedWall` represents a catalytic surface):

```
/*-----* C++ *-----*\
=====
|
```



```

// * * * * *
dimensions      [0 1 -1 0 0 0 0];

internalField    uniform (0 0 0);

boundaryField
{
    heatedWall
    {
        type      stefanFlowVelocity;
        refValue   uniform 0; // dummy value, doesnt affect to BCs
    }
    inlet
    {
        type      fixedValue;
        value      uniform (0.51 0 0);
    }
    ...
}
// * * * * *

```

2.2.4. Setting for temperature field (T) of gas-phase species

For the temperature field T , the internal field and non-reacting boundary conditions can be configured in the same manner as in `reactingFoam`.

At catalytic surfaces, the appropriate thermal boundary condition should be selected according to the physical problem under consideration, such as `fixedValue`, `zeroGradient`, or coupled heat-transfer boundary conditions. When the heat flux generated or consumed by catalytic surface reactions is taken into account, the boundary condition at catalytic surfaces must be specified as `heatFluxFromCatalyticSurfaces`, as illustrated below (e.g., `heatedWall` represents a catalytic surface):

```

/*-----* C++ *-----*\
=====
\\      / F i e l d      | OpenFOAM: The Open Source CFD Toolbox
\\      / O p e r a t i o n | Website:  https://openfoam.org
\\      / A n d              | Version:   8
\\      / M a n i p u l a t i o n |
\*-----*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        volScalarField;
    location     "0";
    object       T;
}
// * * * * *
dimensions      [0 0 0 1 0 0 0];

```

```

internalField    uniform 313;

boundaryField
{
    heatedWall
    {
        type            heatFluxFromCatalyticSurfaces;
        gradient        uniform 0.0;
        value            $internalField;
    }
    inlet
    {
        type            fixedValue;
        value            uniform 313;
    }
    ...
}
// *****

```

2.2.5. Setting for temperature field (*T.catalyst*) of surface species

The file `T.catalyst` represents a temporary temperature field for the solid portion of the catalyst. This file can be created by copying the gas-phase temperature field `T` and subsequently modifying the boundary conditions at catalytic surfaces to `fixedValue` or `zeroGradient`.

It should be emphasized that `T.catalyst` is only required for the program execution and does not influence the simulation results. During the simulation, the temperatures of the solid catalyst surface and the gas phase at the boundary are assumed to be identical. Consequently, the actual gas-phase temperature at the catalytic surface is used in the surface-chemistry calculations.

2.2.6. Setting for pressure field (*p*) of gas-phase species

For the pressure field `p`, the internal field and non-reacting boundary conditions can be configured in the same manner as in `reactingFoam`. For catalytic surface boundaries, no special treatment or additional setup is required.

2.2.7. Setting for pressure field (*p.catalyst*) of surface species

The file `p.catalyst` represents a temporary pressure field for the solid portion of the catalyst. This file can be created by copying the gas-phase pressure field `p` and subsequently

modifying the boundary conditions at catalytic surfaces to appropriate dummy conditions such as `zeroGradient`.

It should be emphasized that `p.catalyst` is only required for program execution and does not influence the simulation results. The actual gas-phase pressure field at the boundary is employed in surface chemistry calculations.

2.3. Settings at constant directory

2.3.1. Setting for `thermophysicalProperties` file

The `thermophysicalProperties` dictionary specifies the thermodynamic, transport, and equation-of-state models employed in the simulation for the gas-phase flow in the `catalystFoam` solver. Its configuration is generally identical to that used in `reactingFoam`, as illustrated in Figure 3. The names of the selected models are critical and must be specified correctly to ensure proper execution of the solver.

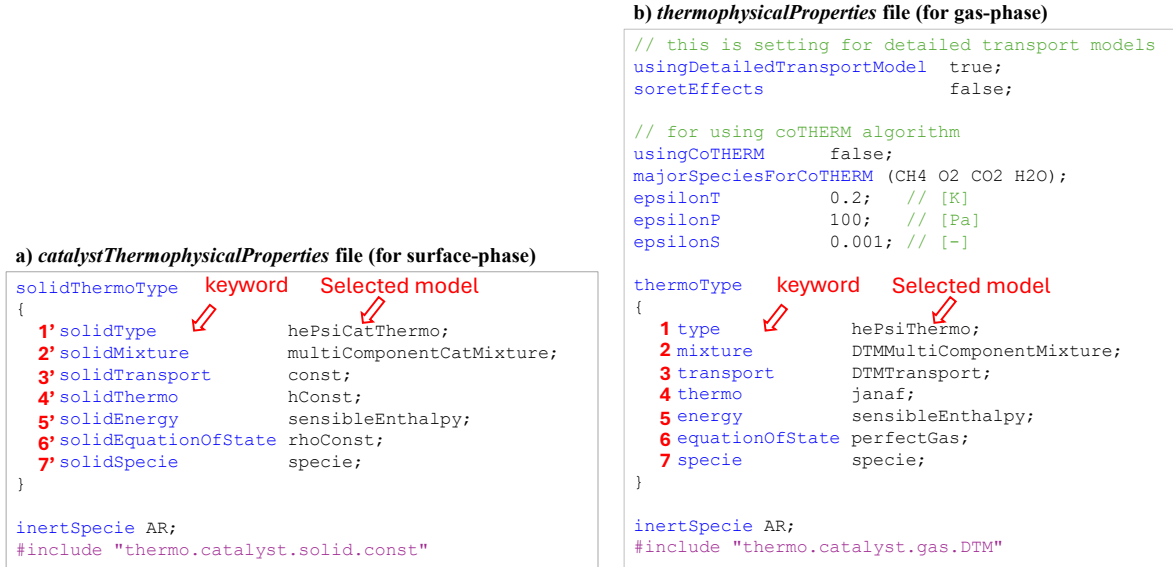


Figure 3: An example settings of the `catalystThermophysicalProperties` file (a) and the `thermophysicalProperties` file (b) for simulations using the `catalystFoam` solver.

Due to the implementation of gas-phase transport models in conjunction with catalytic reacting-flow simulations, only several specific combinations of thermodynamics, transport,

Table 1: Available combinations of gas-phase thermophysical models supported by `catalystFoam`

Options	type	mixture	transport	thermo	energy	equationOfState	specie
a	hePsiThermo	multi ^{a2}	STM ^{a3}	janaf	senE/senI ⁵	perfectGas	specie
b	hePsiThermo	DTMmulti ^{b2}	DTM ^{b3}	janaf	senE/senI ⁵	perfectGas	specie
c	hePsiThermo	FTMmulti ^{c2}	FTM ^{c3}	janaf	senE/senI ⁵	perfectGas	specie
d	hePsiThermo	PPmulti ^{d2}	ppFTM ^{d3}	janaf	senE/senI ⁵	perfectGas	specie

where all abbreviations are:

senE/senI⁵: `sensibleEnthalpy/sensibleInternalEnergy`;

multi^{a2}: `multiComponentMixture`; STM^{a3}: `sutherland`;

DTMmulti^{b2}: `DTMMultiComponentMixture`; DTM^{b3}: `DTMTransport`;

FTMmulti^{c2}: `FTMMultiComponentMixture`; FTM^{c3}: `FTMTransport`;

DTMmulti^{d2}: `DTMMultiComponentMixture`; ppFTM^{d3}: `preProcessingFTMTransport`;

equation-of-state, energy, and mixture models are currently supported by the `catalystFoam` solver, as summarized in Table 1.

Specifically, combination (a) corresponds to the first transport option, in which unity Lewis and Schmidt numbers are assumed, whereas combination (b) corresponds to the second transport option employing the detailed transport model (DTM). When using combination (b), the keyword *usingDetailedTransportModel*, as shown in Fig. 3, must be specified as *true*, since its default value is *false*. When the DTM option is employed, the Soret effect can additionally be enabled or disabled by setting the *soretEffect* (or *soretEffects*) keyword to *true* or *false*, respectively.

To reduce computational cost, the DTM can alternatively be replaced by the fitting transport model (FTM) proposed in [3], or by enabling the coTHERM option (see [3] for details of the coTHERM algorithm). The configuration procedure for enabling the coTHERM algorithm is described in the document entitled *DTLReactingFoam_userGuide_OF8_updated.pdf*, which is provided in the directory `surfaceChemistryFoam-8/documentations/` [1]. If the coTHERM algorithm is not used, no additional coTHERM-related settings are required.

Combination (d) is intended solely for preprocessing purposes when preparing input files for the FTM approach. Additional details regarding the implementation of the DTM and

FTM models, as well as the preparation of the corresponding input files, can be found in the document entitled *DTLReactingFoam_userGuide_OF8_updated.pdf* which is provided in the directory `surfaceChemistryFoam-8/documentations/` [1].

2.3.2. Setting for *catalystThermophysicalProperties* file

The configuration of the `catalystThermophysicalProperties` dictionary file is illustrated in Figure 3-a. For the surface phase, only the thermophysical-model combination shown in this figure is currently supported.

2.3.3. Setting for *chemistryProperties* file

The `chemistryProperties` dictionary specifies the chemistry-related models and solver settings employed in the simulation. Its configuration is generally similar to that used in `reactingFoam`, and no additional settings are required specifically for catalytic reacting-flow simulations. Typical entries include the selection of the chemistry model, activation or deactivation of chemistry calculations, specification of the ODE solver (e.g., `seulex`, `ode`, or `EulerImplicit`), as well as the corresponding absolute and relative tolerances used for stiff chemistry integration.

```
/*-----*- C++ -*-----*\
=====
\\      /  F i e l d      |   OpenFOAM: The Open Source CFD Toolbox
\\      /  O p e r a t i o n   |   Website:  https://openfoam.org
\\      /  A n d               |   Version:   8
\\      /  M a n i p u l a t i o n |
\*-----*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    location     "constant";
    object       chemistryProperties;
}
// * * * * *
chemistryType
{
    solver       ode;
}

chemistry       on;
initialChemicalTimeStep 1e-07;

EulerImplicitCoeffs
{
```

```

        cTauChem          1;
        equilibriumRateLimiter off;
    }

    odeCoeffs
    {
        solver             seulex;
        absTol              1e-8;
        relTol              0.01;
    }

    #include "reactions"
    // *****

```

2.3.4. Setting for *surfaceChemistryProperties* file

The **surfaceChemistryProperties** dictionary specifies the settings associated with catalytic surface chemistry and the numerical treatment of surface reactions. This file is required for simulations involving catalytic reacting surfaces and defines the catalytic regions, surface-chemistry solver configuration, and ODE integration settings used for the surface reaction mechanism.

```

/*-----* C++ *-----*\
=====
\\      / F i e l d           | OpenFOAM: The Open Source CFD Toolbox
\\      / O p e r a t i o n   | Website:  https://openfoam.org
\\      / A n d               | Version:   8
\\      / M a n i p u l a t i o n |
\*-----*/
FoamFile
{
    version      2.0;
    format       ascii;
    class        dictionary;
    location     "constant";
    object       surfaceChemistryProperties;
}
// * * * * *

// identify using particle-resolved model or porous media
onlySurfaceCatalyst true; //-default value is "false"

catalystRegions
{
    catalyticSurfaces (heatedWall); // include catalytic boundary names
    catalyticZones    ( ); // empty list for particle-resolved model
}

catalystToGeometricAreaRatio 30.0; //Fcat/geo, default is 1.0
effectivenessFactor           0.03; //eta, default is 1.0

// setup for selection of ODE solvers
surfaceChemistryType

```

```

{
    solver odeSCHEM;
}

initialChemicalTimeStep 1e-07;

odeSCHEMCoeffs
{
    solver          seulex;
    absTol          1e-8;
    relTol          0.01;
}

#include "reactions.surf"
// *****

```

The keyword `onlySurfaceCatalyst` is used to identify whether the simulation employs a particle-resolved catalytic model or a porous-media catalytic model. When set to `true`, the particle-resolved catalytic model is selected.

The catalytic boundaries (or catalytic zones in the case of porous-media catalytic models) participating in surface reactions are specified within the `catalystRegions` sub-dictionary through the `catalyticSurfaces` (or `catalyticZones`) entry. For example, if the boundary `heatedWall` represents a catalytic surface, its name must be included in the `catalyticSurfaces` list so that surface reactions are activated on that boundary.

The parameter `catalystToGeometricAreaRatio` defines the ratio between the actual catalytic surface area and the geometric surface area used in the simulation, corresponding to $F_{\text{cat}/\text{geo}}$ in the manuscript. The parameter `effectivenessFactor` represents the effectiveness factor, η , introduced in the manuscript. These two factors can be used to account for the influence of surface roughness, internal pore structures, or other effects that increase the effective catalytic surface area relative to the geometric surface area.

The `surfaceChemistryType` sub-dictionary specifies the surface-chemistry solver model employed for integrating the catalytic reaction mechanism. It should be noted that only the `odeSCHEM` option is currently supported. When this option is selected, the `odeSCHEMCoeffs` sub-dictionary must also be provided.

The `odeSCHEMCoeffs` sub-dictionary contains the numerical settings for the ODE solver used to integrate the stiff surface-chemistry equations. Typical settings include the selection of the ODE solver (e.g., `seulex`) together with the corresponding absolute and relative

tolerances that control the accuracy of the chemistry integration.

The parameter `initialChemicalTimeStep` defines the initial time-step size used for surface-chemistry integration.

Finally, the catalytic surface-reaction mechanism in OpenFOAM format is included through the `#include "reactions.surf"` statement, which reads the surface-reaction definitions from the corresponding mechanism file. The procedure for preparing the surface-reaction mechanism file in OpenFOAM format is described in Section 2.5.2.

2.4. Settings at system directory

The files located in the `system` directory contain the numerical and simulation-control settings required for running the case. In general, their configuration is identical to that used in `reactingFoam`, and no special modifications are typically required for catalytic reacting-flow simulations.

2.5. Preparing input thermo and reaction files

2.5.1. Preparing input files for gas-phase

When the first transport option, in which unity Lewis and Schmidt numbers are assumed, is employed, the corresponding thermophysical-model configuration is represented by combination (a) in Table 1. In this case, the preparation of thermodynamic and transport data follows the standard OpenFOAM procedure used for `reactingFoam`.

When the DTM or FTM transport model is employed (i.e., combination (b) or (c) in Table 1), proper preparation of the thermophysical input data becomes essential. This process can be performed using the `DTMchemkinToFoam` and `FTMchemkinToFoam` utilities provided within the framework. Detailed instructions for this procedure can be found in the document entitled *DTLReactingFoam_userGuide_OF8_updated.pdf* which is provided in the directory `surfaceChemistryFoam-8/documentations/` [1].

2.5.2. Preparing input files for surface-phase

For the surface phase, two input files are required. The first is the surface chemical-mechanism file in OpenFOAM format (e.g., `reactions.surf`). The second is the thermo

data file for surface species (e.g., `thermo.catalyst.solid.const`).

The `reactions.surf` file contains the surface-reaction mechanisms and associated kinetic parameters used for heterogeneous surface-chemistry calculations. An example format is shown below:

```
reactions
{
  un-named-reaction-3
  {
    type          irreversibleSurfaceArrheniusSurface;
    reaction       "H(s) + H(s) = Rh(s) + Rh(s) + H2";
    A              5.574e+18;
    beta           0.239;
    Ta             7179.177145;
  }
}
```

The `thermo.catalyst.solid.const` file specifies the thermodynamic and transport properties of the site species. An example format is given below:

```
species
5
(
Rh(s)
H2O(s)
H(s)
OH(s)
O(s)
);

Rh(s)
{
  specie
  {
    molWeight      102.905;
    siteDensity    2.7e-08;
  }
  equationOfState
  {
    rho            8900;
  }
  thermodynamics
  {
    Cp             440;
    Hf             0;
    Cv             440;
  }
  transport
  {
    mu             0;
    Pr             1;
  }
}
...
```

The `reactions.surf` and `thermo.catalyst.solid.const` input files can be generated using `surfChemkinToFoam` utility with the following command:

```
surfChemkinToFoam <input1> <input2> <input3> <output1> <output2>
```

where:

- `input1`: surface-chemistry mechanism file in CHEMKIN format, e.g., `surf.inp`
- `input2`: thermodynamic-data file for surface (site) species in CHEMKIN format, e.g., `surfTherm.dat`
- `input3`: OpenFOAM dictionary file containing the thermodynamic and transport-property definitions for the `const` thermodynamic combination of surface species, e.g., `surfaceThermoDict`
- `output1`: surface-chemistry mechanism file in OpenFOAM format. The file name is arbitrary, e.g., `reactions.surf`
- `output2`: thermodynamic and transport-property file for surface species. The file name is arbitrary, e.g., `thermo.catalyst.solid.const`

The first two input files in CHEMKIN format can be obtained directly from the desired surface-chemistry mechanism available in the literature. In particular, the surface-reaction mechanism file (`surf.inp`) supports several kinetic-constant model keywords implemented in the present framework. The currently supported kinetic-constant models include:

- Arrhenius kinetic constant model (AKC model)
- Coverage-dependent kinetic constant model (CKC model)
- Sticking-coefficient kinetic constant model (SKC model)
- Partial-pressure dependent kinetic constant model (PKC model)

Example definitions for each corresponding kinetic-constant model in the CHEMKIN mechanism file are provided below.

AKC model

$2\text{H(s)} \Rightarrow 2\text{Rh(s)} + \text{H}_2$	5.574E+19	0.239	59.691
CKC model			
$\text{CO(s)} \Rightarrow \text{Rh(s)} + \text{CO}$	1.300E+13	0.295	134.070
COV/CO(s)	0.000E+00	0.000	-47.000/
SKC model			
$2\text{Rh(s)} + \text{H}_2 \Rightarrow 2\text{H(s)}$	3.000E-02	0.000	0.000
STICK			
PKC model			
$2\text{Ru(s)} + \text{H}_2 \Rightarrow 2\text{H(s)}$	2.689E+12	0.000	0.0000
PKC			

The third input file, `surfaceThermoDict`, specifies the thermodynamic and transport properties assigned to the site species for the `const` thermodynamic combination during the conversion process. The required thermodynamic and transport properties of the site species, such as `rho`, `Cp`, and `Cv`, are directly read from the `surfaceThermoDict` file. An example `surfaceThermoDict` file is shown below.

```
surfaceThermoDefaults
{
    // equationOfState
    rho      8900;          // kg/m3

    // transport
    mu       0;             // kg/m/s
    Pr       1;

    // thermodynamics, used only when surfaceThermoModel == const
    Cp       440;           // J/kg/K
    Cv       440;           // J/kg/K
    Hf       0;             // J/kg
}
```

The generated thermo file, `thermo.catalyst.solid.const`, can then be included in the `catalystThermophysicalProperties` dictionary file, as shown in Figure 3. And the generated reaction file, `reactions.surf`, can be included in the `surfaceChemistryProperties` dictionary file, as presented in Section 2.3.4.

At `surfaceChemistryFoam-8/tutorials/surfChemkinToFoam/example/ [1]` directory, an example is provided to demonstrate the use of the `surfChemkinToFoam` utility. It is important to note that using `surfChemkinToFoam` requires only three input files placed in

the case directory. In other words, `0`, `constant`, and `system` directories are not needed in this step.

To generate the `reactions.surf` and `thermo.catalyst.solid.const` file, first go to the case directory by:

```
cd ~/OpenFOAM/yourDirectory/surfaceChemistryFoam-8/tutorials/  
surfChemkinToFoam/example/
```

Then execute the following command in the terminal:

```
// The order of the input files is important and need to be specified  
correctly  
surfChemkinToFoam surf.inp surfTherm.dat surfaceThermoDict reactions.surf  
thermo.catalyst.solid.const
```

After executing the above command, two output files will be generated: `reactions.surf` and `thermo.catalyst.solid.const`. These files are then ready to be used for the `catalystFoam` solver.

Several examples for converting commonly used surface chemical mechanisms are also available in `surfaceChemistryFoam-8/tutorials/surfChemkinToFoam/` directory [1]. In each directory, a script `Allrun` is provided to automate converting files for convenient.

Since the conversion process can sometimes be sensitive to the formatting and consistency of the CHEMKIN input files, an additional document entitled *ManuallyPrepareSurfaceReactionsFile.pdf* is provided in the directory `surfaceChemistryFoam-8/documentations/` [1]. This document is intended to help users debug the conversion process or manually prepare the surface-reaction mechanism file in OpenFOAM format, particularly when the reaction mechanism size is relatively small.

2.6. Running simulation using `catalystFoam`

Once boundary conditions are correctly specified, all required files are in place and the case is ready to run. To start the simulation, first generate the computational mesh using an appropriate utility (e.g., `blockMesh` or `snappyHexMesh`, depending on the problem setup). Alternatively, a mesh generated with external software may be converted into OpenFOAM format. After the mesh has been created, run the solver from the case directory in the same manner as `reactingFoam` by typing:

`catalystFoam`

In the directory `surfaceChemistryFoam-8/tutorials/` [1], a tutorial case featuring a three-sphere catalyst configuration, `NH3decomposition.threeBalls`, is provided to demonstrate the usage of the `catalystFoam` solver. In addition, two validation cases presented in the manuscript are provided. The first case, named `stagnationFlowReactor.Case1`, corresponds to Case 1 presented in Section 5.1.1 of the manuscript. The second case, named `annularFlowReactor`, corresponds to the validation case presented in Section 5.2 of the manuscript. Users are strongly encouraged to study those examples to gain a clear understanding of case setup and execution before applying the solver to their own practical problems.

3. Using surface chemistry library in a custom OpenFOAM solver

3.1. Setup for compilation

To use the surface chemistry library in a custom OpenFOAM solver, the following header files must be included in the solver source code:

```
#include "psiReactionThermo.H" //for gas-phase thermo object
#include "psiReactionCatThermo.H" //for surface-phase thermo object
#include "BasicSurfaceChemistryModel.H" // for surface chemsitry object
```

In the `Make/options` file, the include paths and libraries associated with the surface chemistry and catalyst thermophysical models must be specified correctly, as shown below:

```
EXE_INC = \
    ...
    -I$(LIB_SCHEM8_SRC)/thermophysicalModels/specie/lnInclude \
    -I$(LIB_SCHEM8_SRC)/thermophysicalModels/catalystThermo/lnInclude \
    -I$(LIB_SCHEM8_SRC)/thermophysicalModels/surfaceChemistryModel/
    lnInclude \
    -I$(LIB_SCHEM8_SRC)/catalyticSurfaceBoundaries/lnInclude

EXE_LIBS = \
    -L$(FOAM_USER_LIBBIN) \
    ...
    -lcatspecie \
    -lcatReactionThermophysicalModels \
    -lsurfaceChemistryModel \
    -lcatalyticSurfaceFvPatchFields
```

The following listing presents a complete example of the `Make/options` file used for compiling the `catalystFoam` solver:

```

EXE_INC = \
-I$(LIB_SRC)/finiteVolume/lnInclude \
-I$(LIB_SRC)/meshTools/lnInclude \
-I$(LIB_SRC)/sampling/lnInclude \
-I$(LIB_SRC)/MomentumTransportModels/momentumTransportModels/lnInclude \
-I$(LIB_SCHEM8_SRC)/MomentumTransportModels/compressible/lnInclude \
-I$(LIB_SCHEM8_SRC)/ThermophysicalTransportModels/lnInclude \
-I$(LIB_SCHEM8_SRC)/ThermophysicalTransportModels/psiReactionThermo/lnInclude \
-I$(LIB_SCHEM8_SRC)/thermophysicalModels/specie/lnInclude \
-I$(LIB_SCHEM8_SRC)/thermophysicalModels/reactionThermo/lnInclude \
-I$(LIB_SCHEM8_SRC)/thermophysicalModels/catalystThermo/lnInclude \
-I$(LIB_SCHEM8_SRC)/thermophysicalModels/basic/lnInclude \
-I$(LIB_SCHEM8_SRC)/thermophysicalModels/chemistryModel/lnInclude \
-I$(LIB_SCHEM8_SRC)/thermophysicalModels/surfaceChemistryModel/lnInclude \
-I$(LIB_SRC)/ODE/lnInclude \
-I$(LIB_SCHEM8_SRC)/combustionModels/lnInclude \
-I$(LIB_SCHEM8_SRC)/catalyticSurfaceBoundaries/lnInclude

EXE_LIBS = \
-L$(FOAM_USER_LIBBIN) \
-lODE \
-lfiniteVolume \
-lfvOptions \
-lmeshTools \
-lsampling \
-lcatSpecie \
-lmomentumTransportModels \
-lfluidThermoMomentumTransportModels \
-lthermophysicalTransportModels \
-lpsiReactionThermophysicalTransportModels \
-lreactionThermophysicalModels \
-lcatReactionThermophysicalModels \
-lfluidThermophysicalModels \
-lchemistryModel \
-lsurfaceChemistryModel \
-lcombustionModels \
-lcatalyticSurfaceFvPatchFields

```

3.2. Creating surface chemistry object

A surface-chemistry object, for example `surfaceReaction`, must be created using the `BasicSurfaceChemistryModel` class. The instantiation of this object requires two thermodynamic objects: one for the gas phase, for example `thermo`, and another for the surface phase, for example `catThermo`. Once these thermodynamic objects have been created, the `surfaceReaction` object can be instantiated as follows:

```

// create thermo object
Info<< "Reading thermophysical properties\n" << endl;
autoPtr<psiReactionThermo> pThermo(psiReactionThermo::New(mesh));
psiReactionThermo& thermo = pThermo();

```

```

thermo.validate(args.executable(), "h", "e");
... // create other fields based on thermo object
// create catThermo object
Info<< "Reading thermophysical properties for catalyst\n" << endl;
autoPtr<psiReactionCatThermo> pCatThermo(psiReactionCatThermo::New(mesh));
psiReactionCatThermo& catThermo = pCatThermo();
catThermo.validate(args.executable(), "h", "e");
Info<< "End of creating thermophysical properties for catalyst\n" << endl;

// create surfaceReaction object
Info<< "\nCreating surface chemistry model for catalyst\n" << endl;
autoPtr<BasicSurfaceChemistryModel<psiReactionThermo, psiReactionCatThermo
>>
surfaceReaction
(
    BasicSurfaceChemistryModel<psiReactionThermo, psiReactionCatThermo>::
        New(thermo, catThermo)
);
Info<< "\nEnd of creating surface chemistry model for catalyst\n" << endl;

```

3.3. Updating surface chemistry object

After each time step or iteration, the surface chemistry library must be updated by synchronizing thermodynamic fields from the gas phase and solving the surface reaction system. This is performed as follows:

```

// solve surface reaction kinetics using the selected ODE solver
surfaceReaction->solveSurfaceReactions();

```

In the directory `surfaceChemistryFoam-8/applications/solvers/ [1]`, the source code of the `catalystFoam` is provided to demonstrate the usage of the surface chemistry library. Users are strongly encouraged to study these implementations to gain a clear understanding of the required integration steps before applying the library to their own custom solvers.

3.4. Using surface chemistry with a particle-resolved model

For surface chemistry using a particle-resolved model, the species mass flux and heat flux at catalytic boundaries can be obtained using the following member functions:

```

// specie mass flux [kg/(m2.s)] at a catalytic boundary
const scalarField Rgw = surfaceReaction.speciesFluxAtCatalyticSurfaces(
    speciei, patchi);

// heat flux [J/(m2.s)] at a catalytic boundary
const scalarField Qdotw = surfaceReaction.heatFluxAtCatalyticSurfaces(
    patchi);

```

Here, `speciei` denotes the gas-phase species index, and `patchi` denotes the index of the catalytic boundary patch.

In the directory `surfaceChemistryFoam-8/src/catalyticSurfaceBoundaries/` [1], several boundary-condition models are provided to demonstrate how these fluxes are incorporated into OpenFOAM solvers. Users are strongly encouraged to study these implementations to gain a clear understanding of how the surface chemistry library is coupled with particle-resolved boundary treatments before applying it to their own models.

Users are strongly encouraged to refer to the provided source code examples to gain a thorough understanding of the implementation details and coupling strategy before incorporating the surface chemistry library into their own solvers.

References

- [1] <https://github.com/danhnam11/surfaceChemistryFoam-8>.
- [2] W. Sutherland, The viscosity of gases and molecular force, *Philosophical Magazine* 36 (1893) 507–531.
- [3] D. N. Nguyen, J. H. Lee, C. S. Yoo, DTLreactingFoam: An efficient CFD tool for laminar reacting flow simulations using detailed chemistry and transport with time-correlated thermophysical properties, *Comput. Phys. Commun.* 322 (2026) 110052.